

Python: Basic ML Pipeline Skeleton

```
# — Step 1: Import Libraries —————
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report

# — Step 2: Load Data —————
df = pd.read_csv('data.csv')
print(df.head())          # First 5 rows
print(df.describe())      # Statistical summary
print(df.isnull().sum())  # Check missing values

# — Step 3: Feature / Target Split —————
X = df.drop('target', axis=1)  # Features
y = df['target']               # Labels

# — Step 4: Train / Test Split —————
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# — Step 5: Scale Features —————
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)  # fit + transform on train
X_test  = scaler.transform(X_test)       # transform ONLY on test

# — Step 6: Train Model —————
model = LogisticRegression()
model.fit(X_train, y_train)

# — Step 7: Evaluate Model —————
y_pred = model.predict(X_test)
print('Accuracy:', accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

UNIT 2: Learning by Analogy, Observation & Experimentation

Learning by Analogy

Learning by analogy is based on a simple but powerful idea: if two things are similar in known ways, they are probably similar in unknown ways too.

"If patient A has symptoms similar to patient B, and patient B had disease X, then patient A might also have disease X."

Similarity-Based Reasoning

The core question is: How do we measure SIMILARITY between two data points?

Distance metrics are used to quantify (dis)similarity. The most common:

Euclidean Distance (L2 norm)

```
d(x, y) = sqrt( (x1-y1)^2 + (x2-y2)^2 + ... + (xn-yn)^2 )
```

Example: Point A=(1,2), Point B=(4,6)

```
d = sqrt((4-1)^2 + (6-2)^2) = sqrt(9+16) = sqrt(25) = 5
```

Manhattan Distance (L1 norm)

```
d(x, y) = |x1-y1| + |x2-y2| + ... + |xn-yn|
```

Example: Point A=(1,2), Point B=(4,6)

```
d = |4-1| + |6-2| = 3 + 4 = 7
```

```
# Useful when movement is grid-like (city blocks, chess moves)
```

k-Nearest Neighbors (k-NN)

k-NN is the most fundamental instance-based learning algorithm. It is also called a LAZY LEARNER because it does no training — it simply stores all training data and classifies new points based on the k most similar stored examples.

How k-NN Works

Store all training examples (no 'training' step).

For a new query point x:

- Compute distance from x to every training point.
- Sort training points by distance (ascending).

- c. Select the k nearest neighbors.
- d. For CLASSIFICATION: Predict the majority class among the k neighbors.
- e. For REGRESSION: Predict the average value among the k neighbors.

Numerical Example

Dataset: 5 points with 2 features (x1, x2) and class labels.

ID	x1	x2	Class
P1	1	2	A
P2	2	3	A
P3	3	1	B
P4	5	4	B
P5	4	5	A
Query Q	3	3	?

Step 1: Calculate Euclidean distance from Q=(3,3) to each point:

```

d(Q, P1) = sqrt((3-1)^2 + (3-2)^2) = sqrt(4+1) = sqrt(5) ≈ 2.24
d(Q, P2) = sqrt((3-2)^2 + (3-3)^2) = sqrt(1+0) = 1.00
d(Q, P3) = sqrt((3-3)^2 + (3-1)^2) = sqrt(0+4) = 2.00
d(Q, P4) = sqrt((3-5)^2 + (3-4)^2) = sqrt(4+1) = sqrt(5) ≈ 2.24
d(Q, P5) = sqrt((3-4)^2 + (3-5)^2) = sqrt(1+4) = sqrt(5) ≈ 2.24

```

Step 2: Sort by distance: P2(1.00) → P3(2.00) → P1,P4,P5 (2.24 each)

Step 3: For k=3, nearest neighbors are: P2 (Class A), P3 (Class B), P1 (Class A).

Step 4: Majority vote: A=2, B=1 → Predict Class A for query point Q.

Advantages and Limitations of k-NN

Advantage	Limitation
No training time (lazy learner)	Slow prediction (O(n) per query)
Simple to understand and implement	Memory intensive (stores all data)
Naturally handles multi-class problems	Sensitive to irrelevant features
Adapts well to new data	Curse of dimensionality in high-D space
Non-parametric (no assumptions)	Requires feature normalization

Python Implementation of k-NN

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
import numpy as np
import matplotlib.pyplot as plt

# Load dataset
iris = load_iris()
X, y = iris.data, iris.target

# Split and scale
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Find optimal k using cross-validation
k_range = range(1, 21)
cv_scores = []
for k in k_range:
    knn = KNeighborsClassifier(n_neighbors=k)
    scores = cross_val_score(knn, X_train, y_train, cv=5, scoring='accuracy')
    cv_scores.append(scores.mean())

best_k = k_range[np.argmax(cv_scores)]
print(f'Best k: {best_k}, CV Accuracy: {max(cv_scores):.4f}')

# Train final model
knn = KNeighborsClassifier(n_neighbors=best_k)
knn.fit(X_train, y_train)
print(f'Test Accuracy: {knn.score(X_test, y_test):.4f}')

# Predict new point
new_point = [[5.1, 3.5, 1.4, 0.2]]
new_scaled = scaler.transform(new_point)
print('Predicted class:', iris.target_names[knn.predict(new_scaled)[0]])
```

Learning from Observation & Discovery

This paradigm represents how agents learn by observing their environment and discovering patterns — without a teacher providing explicit labels. This connects directly to exploratory data analysis and unsupervised learning.

Data-Driven Hypothesis Formation

In data-driven hypothesis formation, the machine:

1. Observes raw data (measurements, events, transactions).
2. Identifies recurring patterns, correlations, or anomalies.
3. Forms hypotheses about underlying structure or relationships.
4. Tests hypotheses against new observations.

This is the scientific method applied to machine learning!

Data Generation & Simulation Examples

- **Autonomous Vehicles:** Companies like Waymo use simulators (virtual cities) to generate millions of driving miles to train models safely.
- **Medical AI:** Synthetic patient data generated using GANs to train models when real patient data is scarce.
- **Reinforcement Learning:** The agent's own interactions with the environment generate training data (e.g., AlphaGo playing millions of games against itself).
- **Physics Simulations:** Climate models generate data about scenarios that have not occurred yet.

Python: Data Generation

```
from sklearn.datasets import make_classification, make_regression, make_blobs
import matplotlib.pyplot as plt

# Synthetic classification data
X_cls, y_cls = make_classification(
    n_samples=1000,
    n_features=2,          # 2 features for easy visualization
    n_classes=2,
    n_clusters_per_class=1,
    class_sep=1.5,        # separation between classes
    random_state=42
)

# Synthetic regression data
X_reg, y_reg = make_regression(
    n_samples=500,
    n_features=1,
    noise=20,             # add Gaussian noise
    random_state=42
)
```

```
)

# Synthetic cluster data (for unsupervised)
X_blob, y_blob = make_blobs(
    n_samples=300,
    centers=4,          # 4 clusters
    cluster_std=0.8,
    random_state=42
)

# Visualize blobs
plt.scatter(X_blob[:,0], X_blob[:,1], c=y_blob, cmap='viridis')
plt.title('Synthetic Cluster Data')
plt.show()
```

UNIT 3: Supervised Learning — Regression & Decision Trees

Supervised learning is the backbone of modern ML applications. This unit covers three fundamental algorithms in depth: Linear Regression, Logistic Regression, and Decision Trees. Together, these three algorithms cover the majority of real-world ML tasks.

Linear Regression

Linear Regression is the simplest and most foundational supervised learning algorithm. It models the relationship between a continuous output (dependent variable) and one or more input features (independent variables) as a linear equation.

Simple Linear Regression

With one input feature x and one output y :

```
Model:  $y = \beta_0 + \beta_1 x + \epsilon$ 
```

Where:

```
y = predicted output (dependent variable)
x = input feature (independent variable)
 $\beta_0$  = intercept (value of y when x=0)
 $\beta_1$  = slope (change in y per unit change in x)
 $\epsilon$  = error term (noise)
```

Python: Linear Regression

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
import numpy as np

# Data
X = np.array([[500], [750], [1000], [1250], [1500]])
y = np.array([10, 15, 20, 23, 28])

# Train
model = LinearRegression()
model.fit(X, y)

print('Intercept ( $\beta_0$ ):', model.intercept_)    #  $\approx 1.6$ 
print('Slope ( $\beta_1$ ):', model.coef_[0])        #  $\approx 0.0176$ 

# Predict
```

```

y_pred = model.predict(X)

# Evaluate
mse = mean_squared_error(y, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y, y_pred)
print(f'RMSE: {rmse:.2f}, R²: {r2:.4f}')

# Predict new value
print('Price for 1100 sq.ft:', model.predict([[1100]])[0], 'Lakhs')

```

Logistic Regression

Despite its name, Logistic Regression is a CLASSIFICATION algorithm, not regression. It predicts the PROBABILITY that an input belongs to a particular class (usually binary: 0 or 1).

Python: Logistic Regression

```

from sklearn.linear_model import LogisticRegression
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, confusion_matrix

# Load dataset (binary classification: malignant/benign)
data = load_breast_cancer()
X, y = data.data, data.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = LogisticRegression(C=1.0, max_iter=1000)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[:,1] # Probability of class 1

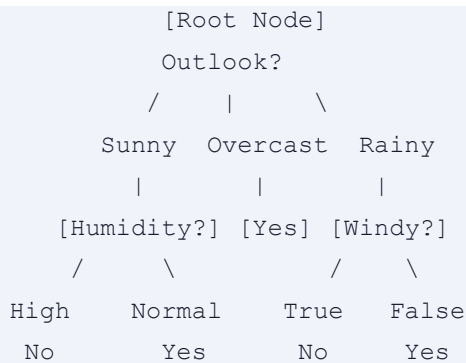
print(classification_report(y_test, y_pred, target_names=data.target_names))
print('Confusion Matrix:')
print(confusion_matrix(y_test, y_pred))

```

Decision Trees

Decision Trees are one of the most intuitive and interpretable ML algorithms. They mimic human decision-making by asking a series of yes/no questions, navigating a tree structure to arrive at a prediction.

Structure of a Decision Tree



- **Root Node:** The first split — the feature that best splits the data.
- **Internal Nodes:** Intermediate decision points — test on a feature.
- **Branches:** Outcomes of the test.
- **Leaf Nodes:** Final prediction (class label or value).

Overfitting in Decision Trees — Pruning

Unpruned decision trees tend to overfit — they memorize every training example. A deep tree can create leaf nodes with just 1 example, perfectly fitting training data but failing on new data.

Pre-Pruning (Early Stopping)

- **max_depth:** Limit tree depth (e.g., max_depth=5). Most important hyperparameter.
- **min_samples_split:** Minimum samples required to split a node (e.g., 20).
- **min_samples_leaf:** Minimum samples in a leaf node (e.g., 5).
- **max_features:** Maximum features to consider for each split.

Post-Pruning (Reduced Error Pruning)

Grow the full tree, then iteratively remove branches that do not improve validation accuracy. Replace subtrees with the majority class leaf if performance is maintained.

Decision Tree ID3 Algorithm

Algorithm ID3(S, Features):

```
If all examples in S have same label → return leaf node with that label
If Features is empty → return leaf with majority label
best_feature = argmax_A IG(S, A) (feature with highest info gain)
Create node split on best_feature
```

```
For each value v of best_feature:
    Sv = subset of S where best_feature = v
    If Sv is empty: add leaf node with majority class of S
    Else: add subtree = ID3(Sv, Features - {best_feature})
Return tree
```

Python: Decision Tree

```
from sklearn.tree import DecisionTreeClassifier, export_text, plot_tree
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt

iris = load_iris()
X, y = iris.data, iris.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)

# Train with max_depth to prevent overfitting
dt = DecisionTreeClassifier(
    criterion='entropy',    # or 'gini'
    max_depth=3,           # key pruning parameter
    min_samples_split=10,
    min_samples_leaf=5,
    random_state=42
)
dt.fit(X_train, y_train)

print('Train Accuracy:', dt.score(X_train, y_train))
print('Test Accuracy: ', dt.score(X_test, y_test))

# Print tree as text rules
print(export_text(dt, feature_names=iris.feature_names))

# Visualize tree
plt.figure(figsize=(12,6))
plot_tree(dt, feature_names=iris.feature_names,
          class_names=iris.target_names, filled=True)
plt.show()

# Feature importance
for name, imp in zip(iris.feature_names, dt.feature_importances_):
    print(f'{name}: {imp:.4f}')
```

Advantages & Disadvantages of Decision Trees

Advantages	Disadvantages
Easy to interpret and visualize	Prone to overfitting (deep trees)
Handles both categorical and numerical data	Unstable — small data change → different tree
No feature normalization needed	Biased toward features with more levels
Captures non-linear relationships	Greedy split may not find global optimum
Handles missing values (C4.5)	High variance model

UNIT 4: Model Evaluation

Building a model is only half the job. Evaluating it correctly is equally important. This unit covers the metrics and techniques needed to honestly assess model performance, choose between models, and avoid misleading results.

The Confusion Matrix

The confusion matrix is the foundation of classification evaluation. It shows the distribution of predictions across all actual classes.

Binary Classification Confusion Matrix

	Predicted: Positive	Predicted: Negative
Actual: Positive	True Positive (TP)	False Negative (FN) — Type II Error
Actual: Negative	False Positive (FP) — Type I Error	True Negative (TN)

- **True Positive (TP):** Model predicted Positive, and it is actually Positive. Correct!
- **True Negative (TN):** Model predicted Negative, and it is actually Negative. Correct!
- **False Positive (FP):** Model predicted Positive, but it is actually Negative. Type I Error (False Alarm).
- **False Negative (FN):** Model predicted Negative, but it is actually Positive. Type II Error (Miss).

Classification Metrics

Accuracy

$$\text{Accuracy} = (\text{TP} + \text{TN}) / (\text{TP} + \text{TN} + \text{FP} + \text{FN})$$

Example: TP=90, TN=850, FP=10, FN=50 → Total=1000

$$\text{Accuracy} = (90+850)/1000 = 940/1000 = 94\%$$

Precision

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

Interpretation: Of all the times the model predicted POSITIVE,
what fraction were actually positive?

High Precision → Low False Positive Rate

Use when: False positives are costly

Example: Spam filter (marking real email as spam is very bad)

Example: TP=90, FP=10 → Precision = 90/100 = 90%

Recall (Sensitivity / True Positive Rate)

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

Interpretation: Of all ACTUAL positives,
what fraction did the model correctly identify?

High Recall → Low False Negative Rate

Use when: Missing a positive is very costly

Example: Cancer detection (missing a cancer patient = fatal miss)

Example: TP=90, FN=50 → Recall = 90/140 ≈ 64.3%

F1 Score — Balancing Precision and Recall

$$\begin{aligned} \text{F1} &= 2 * (\text{Precision} * \text{Recall}) / (\text{Precision} + \text{Recall}) \\ &= 2 * \text{TP} / (2 * \text{TP} + \text{FP} + \text{FN}) \end{aligned}$$

F1 is the harmonic mean of Precision and Recall.

It penalizes extreme imbalance: if either is very low, F1 is low.

Example: Precision=0.90, Recall=0.643

$$\text{F1} = 2 * (0.90 * 0.643) / (0.90 + 0.643) = 2 * 0.579 / 1.543 = 0.750$$

F-beta score (generalized): $F_{\beta} = (1 + \beta^2) * (P * R) / (\beta^2 * P + R)$

$\beta > 1$: emphasizes Recall (use in medical diagnosis)

$\beta < 1$: emphasizes Precision (use in spam detection)

Precision-Recall Tradeoff

Precision and Recall are inversely related — increasing one typically decreases the other:

- Lower classification threshold (e.g., 0.3 instead of 0.5) → More predictions positive → Higher Recall, Lower Precision.
- Higher threshold (e.g., 0.7) → Fewer predictions positive → Lower Recall, Higher Precision.

Specificity & Other Metrics

Metric	Formula	Meaning
Specificity (TNR)	$TN / (TN + FP)$	True Negative Rate — correct rejection rate
FPR (Fall-out)	$FP / (FP + TN)$	False Positive Rate — 1 - Specificity
FNR (Miss Rate)	$FN / (FN + TP)$	False Negative Rate — 1 - Recall
MCC	$(TP*TN - FP*FN) / \text{sqrt}(...)$	Balanced metric even for imbalanced classes

ROC Curve & AUC

The Receiver Operating Characteristic (ROC) curve is a powerful visualization that shows the tradeoff between True Positive Rate (Recall) and False Positive Rate at various classification thresholds.

How to Draw ROC Curve

- Sort predictions by probability (descending).
- For each probability threshold from 1 to 0:
 - Compute $TPR = TP / (TP + FN)$ and $FPR = FP / (FP + TN)$.
 - Plot (FPR, TPR) as a point.
- Connect all points → ROC curve.

AUC (Area Under the ROC Curve)

AUC interpretations:

AUC = 1.0 → Perfect classifier

AUC = 0.9 → Excellent

AUC = 0.7 → Acceptable

AUC = 0.5 → Random classifier (diagonal line) — no skill

AUC < 0.5 → Worse than random (rare, check labels!)

AUC = P(model ranks a random positive higher than a random negative)

Python: ROC Curve

```
from sklearn.metrics import roc_curve, auc, confusion_matrix
from sklearn.linear_model import LogisticRegression
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt

data = load_breast_cancer()
X, y = data.data, data.target
```

```

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
y_proba = model.predict_proba(X_test)[:, 1]

fpr, tpr, thresholds = roc_curve(y_test, y_proba)
roc_auc = auc(fpr, tpr)

plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, color='blue', label=f'ROC curve (AUC = {roc_auc:.4f})')
plt.plot([0,1],[0,1], 'r--', label='Random Classifier')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate (Recall)')
plt.title('ROC Curve')
plt.legend(loc='lower right')
plt.grid(True)
plt.show()

```

Cross-Validation

Cross-validation is a technique to reliably estimate how well your model will generalize to unseen data. It avoids the problem of a lucky/unlucky single train-test split.

Why Cross-Validation?

A single train-test split can be:

- Lucky: test set happens to be easy → overestimates performance.
- Unlucky: test set is hard → underestimates performance.

Cross-validation averages over multiple splits, giving a more reliable estimate.

k-Fold Cross-Validation

Algorithm:

1. Split data into k equal folds (e.g., k=5 or k=10).
2. For i = 1 to k:
 - a. Use fold i as validation set.
 - b. Train on remaining k-1 folds.
 - c. Evaluate on fold i → get score_i.
3. Final CV score = mean(score₁, ..., score_k)

```
(also compute std to assess stability)
```

```
Visualization (k=5):  
Fold 1: [Val][Tr][Tr][Tr][Tr]  
Fold 2: [Tr][Val][Tr][Tr][Tr]  
Fold 3: [Tr][Tr][Val][Tr][Tr]  
Fold 4: [Tr][Tr][Tr][Val][Tr]  
Fold 5: [Tr][Tr][Tr][Tr][Val]
```

- **k=5:** Good balance of speed and reliability. Used commonly.
- **k=10:** More reliable estimate. Sklearn default for `cross_val_score`.
- **k=n (LOOCV):** Leave-One-Out CV. Very reliable but very slow for large datasets.

Stratified k-Fold

In regular k-fold, class distribution may be uneven across folds. Stratified k-fold ensures each fold has the same class proportion as the original dataset. Critical for imbalanced datasets.

Python: Cross-Validation

```
from sklearn.model_selection import cross_val_score, StratifiedKFold, KFold  
from sklearn.tree import DecisionTreeClassifier  
from sklearn.datasets import load_iris  
import numpy as np  
  
iris = load_iris()  
X, y = iris.data, iris.target  
  
dt = DecisionTreeClassifier(max_depth=3, random_state=42)  
  
# Simple k-fold CV  
scores = cross_val_score(dt, X, y, cv=10, scoring='accuracy')  
print(f'CV Accuracy: {scores.mean():.4f} ± {scores.std():.4f}')  
# Stratified k-fold (recommended for classification)  
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)  
strat_scores = cross_val_score(dt, X, y, cv=skf, scoring='f1_weighted')  
print(f'Stratified CV F1: {strat_scores.mean():.4f} ± {strat_scores.std():.4f}')  
# Multiple metrics at once  
from sklearn.model_selection import cross_validate  
results = cross_validate(dt, X, y, cv=5,  
    scoring=['accuracy', 'f1_weighted', 'precision_weighted'])  
for metric, vals in results.items():  
    if metric.startswith('test_'):  
        print(f'{metric}: {vals.mean():.4f}')
```

UNIT 5: Unsupervised Learning, Neural Networks & Overfitting

Unsupervised Learning — Clustering

Clustering is the task of grouping similar data points together without any predefined labels. The goal is to discover natural groupings in data.

K-Means Clustering

K-Means is the most widely used clustering algorithm. It partitions n data points into exactly k clusters, minimizing the total intra-cluster variance (within-cluster sum of squares).

The K-Means Algorithm

10. Choose k (number of clusters).
11. Randomly initialize k cluster centroids (e.g., pick k random data points).
12. Repeat until convergence:
 13. a. ASSIGNMENT STEP: Assign each point to its nearest centroid.
 14. b. UPDATE STEP: Recompute each centroid as the mean of all assigned points.
15. Convergence: No point changes cluster assignment.

Dimensionality Reduction — PCA

Principal Component Analysis (PCA) is the most widely used technique for reducing the number of features in a dataset while preserving as much information (variance) as possible.

Why Reduce Dimensions?

- **Curse of Dimensionality:** As dimensions increase, data becomes sparse. Distances lose meaning. Models need exponentially more data.
- **Visualization:** Reduce to 2D or 3D for plotting.
- **Remove Redundancy:** Correlated features provide no extra information.
- **Speed:** Fewer features → faster training and prediction.
- **Noise Reduction:** Low-variance dimensions often contain noise.

How PCA Works — Step by Step

16. Standardize the data (subtract mean, divide by std for each feature).
17. Compute the Covariance Matrix: $C = \frac{1}{(n-1)} * X^T * X$.
18. Compute Eigenvalues and Eigenvectors of C .
19. Sort eigenvectors by their eigenvalues (largest first) → these are the Principal Components.
20. Select top k eigenvectors → forms the projection matrix W ($d \times k$).
21. Transform data: $Z = X * W$. Z has k features.

Mathematical Understanding

Covariance between features i and j : $\text{cov}(x_i, x_j) = (1/n) * \sum((x_i - \text{mean}_i) * (x_j - \text{mean}_j))$

High cov: features move together (correlated) → redundant information

PCA finds new axes (principal components) that are:

1. Orthogonal (uncorrelated)
2. Maximize variance (most informative directions)

Explained Variance Ratio for component i :

$\text{EVR}_i = \text{eigenvalue}_i / \sum(\text{all eigenvalues})$

If first 2 components explain 95% variance → safe to use just 2D!

Python: PCA

```
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import load_digits
import matplotlib.pyplot as plt
import numpy as np

# Load high-dimensional data (64 features)
digits = load_digits()
X = digits.data    # shape: (1797, 64)
y = digits.target

# Standardize
X_scaled = StandardScaler().fit_transform(X)

# PCA to find explained variance
pca_full = PCA()
pca_full.fit(X_scaled)

# Cumulative explained variance
cumvar = np.cumsum(pca_full.explained_variance_ratio_)
n_components_95 = np.argmax(cumvar >= 0.95) + 1
print(f'Components needed for 95% variance: {n_components_95}')
```



```
# Reduce to 2D for visualization
pca_2d = PCA(n_components=2)
X_2d = pca_2d.fit_transform(X_scaled)
print(f'Variance explained by 2 components: {sum(pca_2d.explained_variance_ratio_):.2%}')
```

```
# Visualize
plt.figure(figsize=(10, 8))
scatter = plt.scatter(X_2d[:,0], X_2d[:,1], c=y, cmap='tab10', alpha=0.6)
plt.colorbar(scatter)
plt.title('PCA: Digits Dataset (64D → 2D)')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.show()
```

Introduction to Neural Networks

Neural Networks are a class of ML models inspired by the human brain's biological neural network. They consist of layers of interconnected artificial neurons (nodes) that transform inputs through learned weights.

The Biological Neuron

A biological neuron: receives signals through DENDRITES, integrates them in the CELL BODY, and fires an output signal through the AXON if the total input exceeds a threshold.

The Artificial Neuron (Perceptron)

The artificial neuron models this process mathematically:

```
1. Receive inputs:  $x = [x_1, x_2, \dots, x_n]$ 
2. Multiply by weights:  $z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$ 
   ( $z = w \cdot x + b$ , where  $b$  = bias)
3. Apply activation function:  $\text{output} = f(z)$ 

 $f(z)$  for the original Perceptron:
 $f(z) = +1$  if  $z \geq 0$ 
 $f(z) = -1$  if  $z < 0$  (step/threshold function)
```

The Perceptron Learning Algorithm

```
Algorithm:
1. Initialize weights  $w = 0$  (or small random values).
2. For each training example  $(x, y)$ :
   a. Compute prediction:  $\hat{y} = f(w \cdot x + b)$ 
   b. If correct ( $\hat{y} = y$ ): no update.
   c. If wrong: update weights:
        $w = w + \alpha * y * x$  ( $\alpha$  = learning rate,  $y$  = true label: +1 or -1)
        $b = b + \alpha * y$ 
3. Repeat for all examples until convergence.
```

Convergence Theorem: Perceptron ALWAYS converges if data is linearly separable.
XOR problem is NOT linearly separable → perceptron CANNOT solve it.

Numerical Example — Perceptron

Problem: Learn AND gate. Inputs: (x1, x2). Labels: 1 if both 1, else 0. Learning rate $\alpha=1$.

x1	x2	y (target)
0	0	0
0	1	0
1	0	0
1	1	1

Initialize: $w_1=0$, $w_2=0$, $b=0$

Epoch 1:

```
(0,0)→0: z=0+0+0=0, f(0)=1 ≠ 0  ERROR
      w1=0+1*(0-1)*0=0, w2=0+1*(0-1)*0=0, b=0+1*(0-1)=-1
(0,1)→0: z=0+0-1=-1, f(-1)=0 = 0  CORRECT
(1,0)→0: z=0+0-1=-1, f(-1)=0 = 0  CORRECT
(1,1)→1: z=0+0-1=-1, f(-1)=0 ≠ 1  ERROR
      w1=0+1*1*1=1, w2=0+1*1*1=1, b=-1+1*1=0
... (continues over multiple epochs until convergence)
```

Final converged: $w_1=1$, $w_2=1$, $b=-1.5$

```
Verify: (1,1): z=1+1-1.5=0.5 → 1 ✓
      (1,0): z=1+0-1.5=-0.5 → 0 ✓
```

Multi-Layer Perceptron (MLP)

To solve non-linearly separable problems (like XOR), we stack multiple layers of neurons:

MLP Architecture:

[Input Layer] → [Hidden Layer 1] → [Hidden Layer 2] → ... → [Output Layer]

Each layer:

- Takes previous layer outputs as inputs
- Applies weighted sum + bias + activation function
- Outputs to next layer

Notation:

```
L = number of layers
n_l = number of neurons in layer l
W_l = weight matrix for layer l (n_l × n_{l-1})
b_l = bias vector for layer l
a_l = activation output of layer l
```

Forward Pass:

```
z_1 = W_1 * a_{l-1} + b_1
a_1 = f(z_1)  (element-wise activation)
```

Activation Functions

Function	Formula	Range	Use Case
Sigmoid	$1/(1+e^{(-z)})$	(0,1)	Output layer (binary classification)
Tanh	$(e^z - e^{-z})/(e^z + e^{-z})$	(-1,1)	Hidden layers (centered at 0)
ReLU	$\max(0,z)$	$[0,\infty)$	Hidden layers (default modern choice)
Leaky ReLU	$\max(0.01z, z)$	$(-\infty,\infty)$	Avoids dying ReLU problem
Softmax	$e^{z_i} / \sum(e^{z_j})$	(0,1)	Output layer (multi-class)

ReLU is the default activation for hidden layers in modern neural networks because it avoids the vanishing gradient problem and is computationally cheap.

Backpropagation (Brief)

Backpropagation is the algorithm used to train neural networks. It computes the gradient of the loss function with respect to each weight using the chain rule, then updates weights via gradient descent.

Forward pass: Compute predictions and loss.

Backward pass: Compute gradients using chain rule:

$$\partial L / \partial W_1 = \partial L / \partial a_1 * \partial a_1 / \partial z_1 * \partial z_1 / \partial W_1$$

Weight update: $W_1 = W_1 - \alpha * \partial L / \partial W_1$

Vanishing Gradient Problem:

Sigmoid/Tanh gradients $\rightarrow 0$ for large $|z|$

Deep networks: gradients become tiny in early layers

$\rightarrow$ Weights stop updating $\rightarrow$ Learning freezes

Solution: Use ReLU, Batch Normalization, Residual connections

Python: MLP with Scikit-learn

```
from sklearn.neural_network import MLPClassifier
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score

digits = load_digits()
X, y = digits.data, digits.target
```

```

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

mlp = MLPClassifier(
    hidden_layer_sizes=(100, 50), # 2 hidden layers: 100 and 50 neurons
    activation='relu',           # ReLU activation
    solver='adam',               # Adam optimizer
    max_iter=500,
    learning_rate_init=0.001,
    random_state=42
)

mlp.fit(X_train, y_train)
y_pred = mlp.predict(X_test)
print(f'MLP Test Accuracy: {accuracy_score(y_test, y_pred):.4f}')
print(f'Training loss curve: {mlp.loss_curve_[-5:]})

```

Overfitting & Underfitting

Overfitting and underfitting represent the two failure modes in machine learning. Understanding and preventing them is one of the most critical skills in ML.

Visual Intuition

Underfitting:	Good Fit:	Overfitting:
<pre> y / / / (too simple) / +-----x </pre>	<pre> y * * * * * * (just right) * * +-----x </pre>	<pre> y * * * * * * * * (memorizes) * +-----x </pre>
Train error: HIGH	Train error: LOW	Train error: VERY LOW
Test error: HIGH	Test error: LOW	Test error: HIGH
Model is too simple.	Good generalization.	Model memorized noise.

